

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

Debugging Signed Integer Addition Using 2's Complement

Problem: A program performing signed integer addition using 2's complement produces incorrect results when adding large values such as +120 and +50 in 8-bit representation. The task is to debug the binary computation, identify overflow and propose a solution to detect and handle it correctly.

Identification of Errors / Bugs

The primary error is signed overflow. An 8-bit signed 2's complement number can represent only -128 to $+127$. Although +120 and +50 are individually valid, their mathematical sum is +170, which is outside this range.

$$120 + 50 = 170$$

Therefore, the 8-bit arithmetic unit cannot represent +170 as a signed 8-bit value. If the overflow condition is ignored, the resulting bit pattern is interpreted as a negative number, producing an incorrect application-level result.

Debugging Approach & Analysis

First convert the operands to 8-bit binary.

$$120 = 01111000$$

$$50 = 00110010$$

Add them:

$$\begin{array}{r} 01111000 \\ + 00110010 \\ \hline 10101010 \end{array}$$

The result has $\text{MSB} = 1$. In signed 2's complement, this pattern is negative.

$$10101010 \rightarrow \text{invert} \rightarrow 01010101 \rightarrow +1 \rightarrow 01010110$$

$$01010110 = 86$$

Thus 10101010 represents -86 , whereas the mathematical result is $+170$. This mismatch is the evidence of signed overflow.

Root Cause

- Both input operands are positive, but their sum is larger than $+127$.
- The 8-bit result wraps around into the negative region.
- The program fails to test the signed overflow condition before accepting the result.
- The extra carry alone is not a sufficient signed-overflow test; the sign relationship of operands and result must also be considered.

Correctness of Debugged Solution

For signed addition, overflow occurs when two operands with the same sign produce a result with the opposite sign. Here both operands are positive and the result has sign bit 1.

01111000 $\rightarrow$ positive

00110010 $\rightarrow$ positive

10101010 $\rightarrow$ negative

Therefore:

Overflow = 1

A hardware implementation can also detect overflow using:

$V = \text{Carry into MSB XOR Carry out of MSB}$

The program should not treat 10101010 as the valid result $+170$ in an 8-bit signed variable.

Optimization / Improvement

- Use a wider integer representation, such as 16-bit arithmetic, when values can exceed the 8-bit signed range.
- Set an overflow flag in the arithmetic unit.
- Use saturation arithmetic when the application requires limiting the output to $+127$ instead of wrap-around.
- Generate an exception or error status so higher-level software can respond.
- Perform range checking when the input limits are known.

For 16-bit arithmetic:

00000000 01111000

+00000000 00110010

00000000 10101010

This represents +170 correctly.

Testing and Documentation

Test	Expected	Debugged behavior
20 + 30	50	50 – Pass
100 + 20	120	120 – Pass
120 + 50	170	Overflow flag – Pass
127 + 1	Overflow	Overflow flag – Pass
–50 + –60	–110	–110 – Pass

Conclusion: The bug is signed overflow. Correct overflow detection prevents an invalid wrapped result from being interpreted as a valid signed answer.

- A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.**

Debugging Ripple Carry Adder Delay and Optimising Using CLA

Problem: A processor using a ripple carry adder shows significant delay in arithmetic operations in a real-time system. The task is to examine carry propagation, identify the bottleneck and replace or optimize the design using a carry look-ahead adder.

Identification of Error / Bottleneck

The main issue is not incorrect arithmetic; it is excessive propagation delay. In a ripple carry adder, each full-adder stage waits for the carry generated by the previous stage.

$C_0 \rightarrow FA_0 \rightarrow C_1 \rightarrow FA_1 \rightarrow C_2 \rightarrow FA_2 \rightarrow C_3 \rightarrow FA_3 \rightarrow C_4$

Therefore, the worst-case delay increases as the number of bits increases.

Debugging Approach & Analysis

For every bit position:

Generate: $G_i = A_i \cdot B_i$

Propagate: $P_i = A_i \text{ XOR } B_i$

Carry: $C_{i+1} = G_i + P_i C_i$

The ripple architecture passes the carry sequentially. If the least significant stage changes, the final carry may have to wait for every intermediate stage.

Root Cause

- Serial dependency between full-adder stages.
- Each stage waits for the previous carry.
- Longer word sizes produce longer worst-case carry paths.
- High-frequency real-time systems cannot tolerate unnecessary arithmetic delay.

Correctness of Debugged Solution

Replace the ripple structure with a carry look-ahead structure. The carry equations are:

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

$$S_i = P_i \text{ XOR } C_i$$

These equations allow carries to be derived from the original input information instead of waiting for the previous full-adder to finish.

Optimization Techniques Applied

Feature	Ripple Carry	Carry Look-Ahead
Carry generation	Sequential	Look-ahead/parallel logic
Delay	Higher	Lower
Speed	Lower	Higher
Logic complexity	Lower	Higher
Real-time suitability	Lower	Higher
Processor application	Basic ALU	High-speed ALU

The optimization trades additional combinational hardware for reduced carry delay. Larger processors can also use hierarchical CLA blocks to balance speed, area and wiring complexity.

Testing and Documentation

A	B	Expected	CLA Output	Status
0011	0010	0101	0101	Pass
0101	0011	1000	1000	Pass
1111	0001	0000 + carry	Correct	Pass

Conclusion: The performance bottleneck is ripple carry propagation. CLA maintains arithmetic correctness while reducing carry waiting time, making it more appropriate for high-performance real-time arithmetic.

- A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.**

Debugging Booth's Multiplication for Negative Operands

Problem: A multiplication unit implementing Booth's algorithm produces incorrect results for some negative operands. The debugging must focus on bit-pair checking, arithmetic shifts and sign extension, followed by corrections for accurate signed multiplication.

Identification of Errors / Bugs

- Incorrect Q0 and Q-1 bit-pair checking.
- Using logical right shift instead of arithmetic right shift.
- Failure to sign-extend the accumulator.
- Incorrect 2's complement conversion of negative operands.
- Incorrect initialization of Q-1.
- Incorrect number of iterations.
- Incorrect choice between $A + M$ and $A - M$.

Debugging Approach & Analysis

Booth's algorithm examines Q0 and Q-1 together:

Q0 Q-1	Required operation
00	No operation
01	$A = A + M$
10	$A = A - M$

11	No operation
----	--------------

After the selected arithmetic operation, the combined A-Q-Q-1 registers must undergo an arithmetic right shift. The sign bit of A must be copied into the new MSB.

Debugging Example: $-6 \times +5$

$$M = -6 = 1010$$

$$Q = +5 = 0101$$

$$A = 0000$$

$$Q-1 = 0$$

Cycle 1: $Q_0Q_{-1} = 10$, so $A = A - M$.

$$0000 - 1010 = 0110$$

$$\text{Arithmetic shift} \rightarrow A=0011, Q=0010, Q_{-1}=1$$

Cycle 2: $Q_0Q_{-1} = 01$, so $A = A + M$.

$$0011 + 1010 = 1101$$

$$\text{Arithmetic shift} \rightarrow A=1110, Q=1001, Q_{-1}=0$$

Continue the same process for the remaining cycles. The final product is:

$$AQ = 11100010$$

$$11100010 = -30$$

The result agrees with ordinary decimal multiplication: $-6 \times 5 = -30$.

Root Cause Example – Wrong Shift

A logical shift fills the new MSB with zero. This is incorrect for a negative signed partial product.

$$\text{Logical: } 1110 \rightarrow 0111$$

$$\text{Arithmetic: } 1110 \rightarrow 1111$$

Therefore, using a logical shift can destroy the sign information and cause incorrect products.

Correctness of Debugged Solution

- Represent negative operands using the correct 2's complement width.
- Initialize Q_{-1} to zero.
- Use the exact Booth decision table.

- Perform arithmetic right shifts after every cycle.
- Sign-extend the accumulator.
- Perform the required number of iterations.
- Verify the final AQ product against decimal multiplication.

Optimization

Booth encoding reduces unnecessary additions for runs of consecutive 1s in the multiplier. Hardware implementations can also use modified Booth techniques for higher-radix multiplication, but the basic correction required for this question is correct signed bit handling and arithmetic shifting.

Testing

Multiplicand	Multiplier	Expected
−6	+5	−30
+6	+5	+30
−6	−5	+30
+6	−5	−30
−4	+3	−12

Conclusion: Incorrect signed multiplication is commonly caused by bit-pair errors, wrong shifting or sign-extension mistakes. Correct Booth control and arithmetic shifts restore accurate signed multiplication.

- 4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.**

Optimising Restoring Division Using Non-Restoring Division

Problem: An embedded system using restoring division is inefficient because repeated restoration steps increase execution time. The task is to identify unnecessary operations and optimize the implementation using non-restoring division.

1. Identification of Error / Bottleneck

The restoring algorithm is not mathematically incorrect. Its inefficiency comes from repeatedly restoring a negative trial remainder.

After shifting and subtracting the divisor, if the result becomes negative, restoring division performs an additional addition to return to the previous partial remainder.

Shift → Subtract M → Negative? → Yes → Restore by adding M

This additional restoration is the main source of unnecessary work.

Debugging Approach & Analysis

1. Track the partial remainder at every iteration.
2. Record every trial subtraction.
3. Identify iterations where the trial remainder becomes negative.
4. Count how often restoration is performed.
5. Compare the number of arithmetic operations with a non-restoring implementation.

Example: $13 \div 3$

Dividend = 13 = 1101

Divisor = 3 = 0011

Expected quotient = 4

Expected remainder = 1

The mathematical verification is:

$$13 = (3 \times 4) + 1$$

Root Cause

The repeated restore operation increases the number of additions/subtractions and control decisions. In a processor or embedded controller, these extra operations increase latency and can reduce throughput.

Correctness of Optimized Solution

Non-restoring division avoids immediate restoration. Instead:

If $A \geq 0$: shift and subtract M

If $A < 0$: shift and add M

A negative partial remainder is therefore retained temporarily and corrected through the next operation. At the end, if the final remainder is negative, a final correction by adding M is performed.

Optimization Applied

Feature	Restoring	Non-Restoring
Negative partial remainder	Immediately restored	Temporarily retained
Repeated restore	Yes	No
Arithmetic operations	Generally more	Generally fewer
Latency	Higher	Lower
Efficiency	Lower	Higher

The main optimization is eliminating unnecessary restoration operations. This improves the efficiency of the division unit while maintaining the same quotient and remainder.

Testing

Division	Expected Quotient	Expected Remainder
$13 \div 3$	4	1
$20 \div 4$	5	0
$15 \div 2$	7	1
$10 \div 3$	3	1

Conclusion: The inefficiency comes from repeated restoration. Non-restoring division uses the sign of the partial remainder to choose the next operation and therefore reduces unnecessary work.

- 5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.**

Debugging IEEE 754 Addition Accuracy

Problem: A scientific application using IEEE 754 single precision gives inaccurate results when adding very small and very large numbers. The task is to analyze exponent alignment, normalization and rounding errors and propose improvements such as double precision or algorithmic adjustments.

Identification of Errors / Issues

The main issue is limited floating-point precision. Single precision uses 32 bits: 1 sign bit, 8 exponent bits and 23 fraction bits. When two numbers have very different magnitudes, exponent alignment can shift the smaller significand so far to the right that important low-order bits are lost.

Debugging Approach & Analysis

1. Inspect the sign, exponent and fraction of both operands.
2. Compare their exponents.
3. Align the smaller exponent to the larger exponent.
4. Inspect which fraction bits are shifted out.
5. Perform significand addition/subtraction.
6. Normalize the result.
7. Apply rounding.
8. Compare the stored result with a higher-precision reference.

Exponent Alignment

Consider normalized values with different exponents:

$$X = 1.1000 \times 2^8$$

$$Y = 1.1000 \times 2^2$$

To add them, Y must be aligned to exponent 8:

$$Y = 0.0001100 \times 2^8$$

The smaller operand has undergone a large right shift. In a finite-precision representation, low-order bits may be discarded or rounded.

Root Cause

- Single precision has a limited fraction field.
- Large exponent differences cause extensive right shifts.
- Low-order information can be lost during alignment.
- Rounding changes the stored result when extra bits are present.
- Repeated operations can accumulate rounding errors.
- A very small number may have little or no effect when added to a much larger number.

Correctness of Debugged Solution

The first correction is to use a numerical format with sufficient precision. Double precision provides 52 explicit fraction bits compared with 23 in single precision.

Feature	Single Precision	Double Precision
Total bits	32	64
Exponent bits	8	11
Fraction bits	23	52
Precision	Lower	Higher
Scientific numerical use	Moderate	Higher accuracy

Optimization Techniques Applied

- Use double precision for calculations requiring greater numerical accuracy.
- Use compensated summation for repeated addition of values with different magnitudes.
- Choose a suitable summation order when mathematically permissible.
- Scale data when appropriate to avoid extreme magnitude differences.
- Use a higher-precision reference implementation during validation.

These techniques improve numerical quality, though double precision may require additional memory and computation.

Testing and Validation

Test	Purpose	Expected observation
Large + large	Check ordinary precision	Accurate within format precision
Small + small	Check small-value handling	Accurate within format precision
Large + very small	Check absorption/rounding	Small contribution may be lost
Same tests in double precision	Compare accuracy	Usually improved precision
Repeated summation	Measure accumulated error	Compensated method can reduce error

Conclusion: The issue is primarily numerical precision rather than a simple syntax error. Using double precision and suitable numerical algorithms can reduce loss of significance and improve scientific computation accuracy.